cwilliams, garias@utec.edu.pe
CS4054: Networks | 26-1
October 17, 2026

Ex 4: Transport Layer – TCP

Part I: Theoretical & Analytical Questions

Question 1. AIMD vs. AIAD Fairness:

a. Show that the TCP AIMD (Additive Increase, Multiplicative Decrease) algorithm guarantees a fair bandwidth allocation between two competing flows starting from an initial operating point $P_0(x_1, x_2)$. Use the two-dimensional phase-plane diagram below as a reference and provide concrete example values to demonstrate step-by-step convergence toward the fairness line.

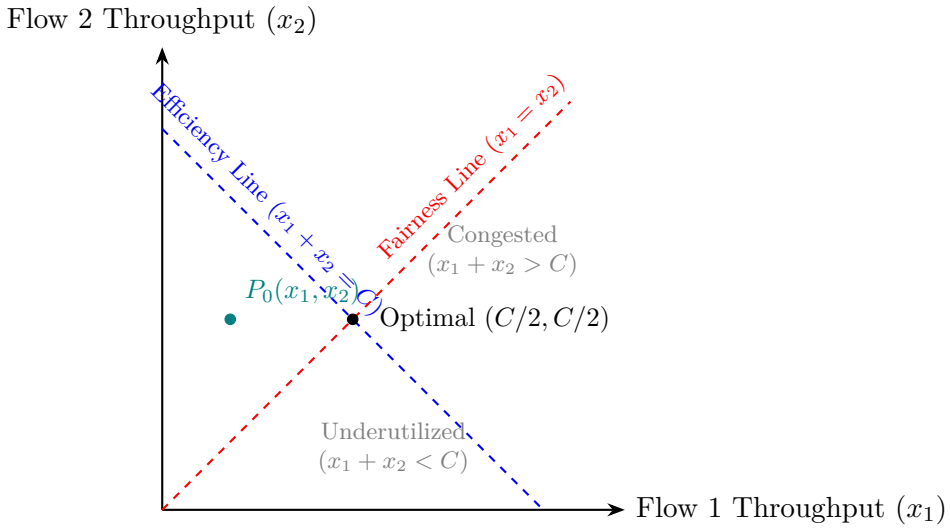


FIGURE 1. Chiu-Jain phase plane for two competing TCP flows.

b. Suppose that, instead of a multiplicative decrease, TCP reduced the congestion window size by a constant amount upon detecting loss. Would the resulting AIAD (Additive Increase, Additive Decrease) algorithm converge toward an equal-share allocation? Justify your answer using a phase-plane diagram.

Question 2. GBN, SR, and TCP (No Delayed ACK): Assume that the timeout values for Go-Back-N (GBN), Selective Repeat (SR), and TCP are sufficiently long such that five consecutive data segments and their corresponding ACKs can be transmitted and received (if not lost in the channel) by the receiving host (Host B) and the sending host (Host A), respectively, before any timer expires.

Suppose Host A sends five data segments (numbered 1 through 5) to Host B, and the second segment (S_2) is lost during the initial transmission, as shown below. Ultimately, all five data segments are correctly received by Host B.

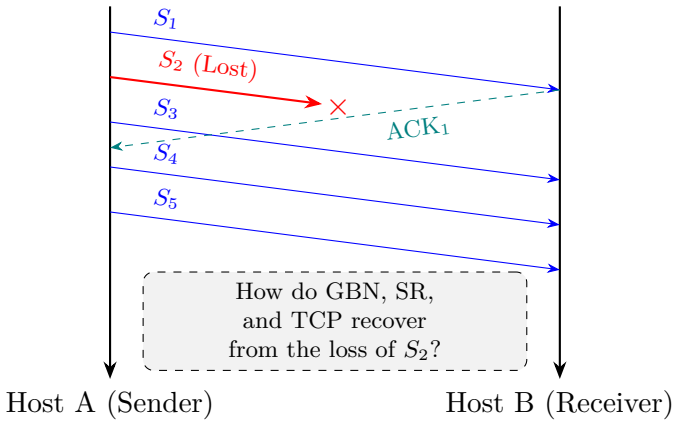


FIGURE 2. Initial transmission of 5 segments with S_2 lost in transit.

a. How many total segments has Host A sent, and how many total ACKs has Host B sent? What are their respective sequence and acknowledgment numbers? Answer this question for each of the three protocols (assume TCP uses Fast Retransmit upon receiving 3 duplicate ACKs).
b. If the timeout values for all three protocols are much greater than 5 RTT, which protocol successfully delivers all five data segments in the shortest time interval? Justify your answer.

Question 3. NAK-Only Protocol: Consider a reliable data transfer protocol that uses exclusively negative acknowledgments (NAKs) rather than positive acknowledgments (ACKs).

a. Suppose the sender transmits data only infrequently. Would a NAK-only protocol be preferable to an ACK-based protocol in this scenario? Explain why or why not (consider how the receiver detects a lost packet).
b. Now suppose the sender has a large, continuous stream of data to transmit and the end-to-end connection experiences very few packet losses. In this second case, would a NAK-only protocol be preferable to an ACK-based protocol? Justify your answer in terms of bandwidth overhead and error recovery delay.

Question 4. TCP RTT Estimation and Karn's Algorithm: Suppose a TCP sender measures the following three consecutive SampleRTT values: 106 ms, 120 ms, and 140 ms. Prior to these measurements, the initial state is EstimatedRTT = 100 ms and DevRTT = 5 ms. Assume the standard TCP weighting factors $\alpha = 0.125$ and $\beta = 0.25$.

a. Compute the updated EstimatedRTT, DevRTT, and the resulting TimeoutInterval after each of the three SampleRTT measurements using the standard TCP equations:

$$\text{EstimatedRTT} = (1-\alpha)\cdot\text{EstimatedRTT} + \alpha\cdot\text{SampleRTT}$$
$$\text{DevRTT} = (1-\beta)\cdot\text{DevRTT} + \beta\cdot|\text{SampleRTT} - \text{EstimatedRTT}|$$
$$\text{TimeoutInterval} = \text{EstimatedRTT} + 4\cdot\text{DevRTT}$$

b. Explain why TCP avoids measuring SampleRTT for retransmitted segments (**Karn's Algorithm**). What ambiguity arises if the sender updates EstimatedRTT using the acknowledgment of a retransmitted segment?

Question 5. TCP Congestion Control: Tahoe vs. Reno: Consider a TCP connection with an initial congestion window of cwnd = 1 MSS and a slow-start threshold of ssthresh = 8 MSS. Assume that a **triple duplicate ACK** event occurs at transmission round 4, and a **timeout** event occurs at transmission round 9.

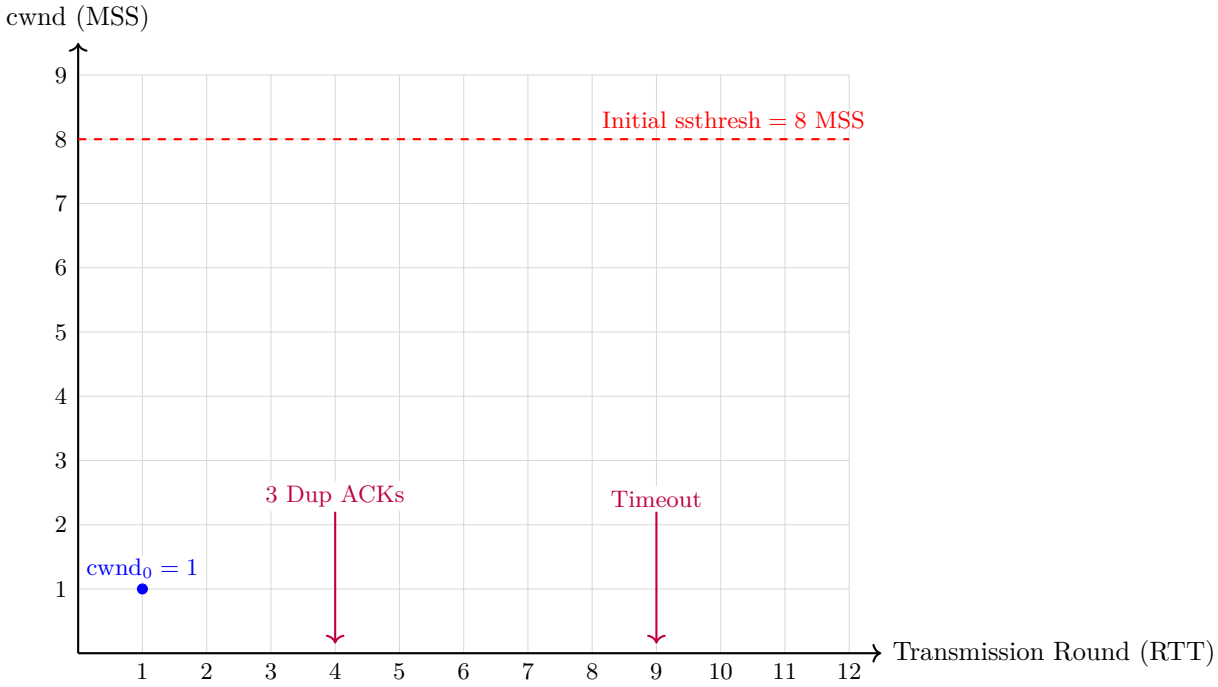


FIGURE 3. Reference grid for tracing cwnd and ssthresh from round 1 to round 12.

a. Trace the values of cwnd and ssthresh from round 1 to round 12 for both **TCP Tahoe** and **TCP Reno**. Present your results in a comparative table and plot cwnd on the reference grid above.
b. Why does TCP Reno recover differently from a triple duplicate ACK compared to a timeout, whereas TCP Tahoe treats both events identically?

Part II: Wireshark & Programming Lab Exercises

Question 6. TCP Connection Analysis with Wireshark:

Objective. Capture and analyze a real TCP connection established between your computer and a web server. Use Wireshark and a web browser to observe connection establishment, data transfer, and connection termination.

Instructions.

1) Open Wireshark and select your active network interface.
2) Apply a capture or display filter for HTTP/HTTPS TCP traffic (e.g., `tcp.port == 80` or `tcp.port == 443`).
3) Visit a lightweight webpage such as `http://example.com` or `http://httpbin.org/get`.
4) Stop the capture once the page has fully loaded.
5) Isolate the TCP stream corresponding to the connection between your host and the server.

Questions.

1) What were the IP addresses and port numbers used by the client and the server?
2) Which specific packet numbers correspond to the Three-Way Handshake? Include a screenshot.
3) What header fields and flags confirm that the TCP connection was successfully established?
4) How many data-carrying TCP segments were transmitted before the connection closed?
5) Which packets indicate the connection termination sequence (`FIN` and `ACK`)?

6) What was the initial and maximum receiver window size (`Win`) advertised during the capture?
7) What differences do you observe in payload visibility and handshake overhead between HTTP (port 80) and HTTPS (port 443) traffic in Wireshark?

Question 7. Local TCP Client–Server Communication in Python:

Objective. Design and analyze a custom TCP communication system using Python sockets to inspect how application-level delays and sequential transmissions interact with TCP mechanisms.

Instructions.

1) Implement a TCP server in Python capable of handling multiple consecutive client messages within a persistent connection.
2) Upon receiving a message from the client, the server must wait for 2 seconds (simulating processing/network delay) before sending back an application-layer acknowledgment string.
3) Implement a TCP client that:
   a. Establishes a connection with the server,
   b. Sends 3 separate text messages (waiting 1 second after receiving each reply before sending the next),
   c. Waits for the server's reply after each message,
   d. Measures and displays the elapsed application-level Round Trip Time (RTT) for each message.
4) Run the experiment first on the loopback interface (`127.0.0.1`), and then—if possible—between two machines on the same local network.
5) Capture the packet exchange in Wireshark using the display filter:

```
tcp.port == <chosen_port>
```

Questions.

1) How many TCP segments were exchanged per application message (distinguish between transport-layer TCP ACKs and application-layer replies)?
2) How do the Sequence (`Seq`) and Acknowledgment (`Ack`) numbers evolve across the 3 sequential messages?
3) Why does Wireshark send a transport-layer TCP `ACK` almost immediately, even though the Python server waits 2 seconds before sending its reply?
4) Compare the application-layer RTT measured by your Python client with the TCP RTT reported by Wireshark (`tcp.analysis.ack_rtt`). Why do they differ?
5) Observe the TCP window size throughout the communication. Does it change between messages, and what does it represent?
6) What happens at the packet level (`RST` or `FIN`) if you abruptly terminate the server process (`Ctrl+C`) while the client is still connected? How does TCP handle this compared to UDP?
7) List the exact chronological sequence of TCP flags (`SYN`, `ACK`, `PSH`, `FIN`) observed from start to finish and explain the role of the `PSH` flag.

Question 8. (Group) Sending a `.txt` File Between Two Laptops Using TCP:

Objective. Implement a TCP client–server file transfer system between two hosts on the same wireless network and evaluate how TCP segments large payloads.

Instructions.

1) Connect both laptops to the same Wi-Fi network (e.g., a mobile hotspot).
2) Identify the private IP address of the laptop acting as the server.
3) Write a Python TCP server that listens on port `5000`, receives a byte stream, and saves it locally as `received.txt`.
4) Write a Python TCP client that connects to the server and transmits a text file named `message.txt` (ensure the file size is at least 100 KB so multiple TCP segments are generated).
5) Capture the transmission in Wireshark filtering by `tcp.port == 5000`.
6) Verify file integrity by comparing the SHA-256 or MD5 checksum of `message.txt` and `received.txt`.

Questions.

1) What were the IP addresses of the server and the client during the connection? Include a screenshot of the capture.
2) How many total TCP data segments were generated during the file transfer, and what was the Maximum Segment Size (MSS) negotiated during the handshake?
3) What was the payload size of the final data segment transmitted?
4) Identify the Three-Way Handshake and the Connection Termination packets in your capture.
5) Were there any retransmissions or duplicate ACKs during the transfer (filter with `tcp.analysis.flags`)? If so, what triggered them?
6) What was the maximum calculated window size observed during the connection?
7) How does each acknowledgment number (`Ack`) relate to the cumulative file bytes received so far?
8) Express the mathematical relationship between total file size, MSS, and the number of TCP data segments transmitted.
9) How does wireless interference or packet loss on a Wi-Fi hotspot impact TCP throughput?
10) What differences in RTT stability, MSS, and retransmission rate would you expect if both laptops were connected via Gigabit Ethernet instead of Wi-Fi?

Question 9. Visualizing TCP Throughput, Sequence Numbers, and Congestion Control in Wireshark:

Objective. Use Wireshark's built-in TCP Stream Graphs to visually analyze the evolution of sequence numbers, in-flight bytes, throughput, and flow/congestion control during a large file download.

Instructions.

1) Start a Wireshark capture on your main network interface.
2) Download a moderately sized file (10 MB to 50 MB) over HTTP or HTTPS from an external server (for example, using `curl` or a browser from a public mirror or speed-test file server).
3) Stop the capture once the download finishes and select one of the main data packets belonging to the download stream.
4) Navigate to `Statistics -> TCP Stream Graphs` in Wireshark and generate the following plots:
   - **Time Sequence (Stevens)**
   - **Throughput**
   - **Window Scaling**

Questions.

1) Include a screenshot of the **Time Sequence (Stevens)** graph. What does the slope of the curve represent, and how can you visually identify the *Slow Start* phase at the beginning of the connection?
2) Did the slope flatten at any point during the download? Explain how a flat horizontal plateau on a Stevens graph corresponds to packet loss or receiver stalls.
3) Apply the display filter `tcp.analysis.retransmission or tcp.analysis.fast_retransmission`. Did the connection experience any losses? If so, did it recover via Fast Retransmit or a Timeout?
4) Inspect the **Window Scaling** graph (which compares the Receiver Window `rwnd` against the estimated bytes in flight). Was the download bottlenecked by the receiver's buffer (`rwnd`) or by network congestion (`cwnd`)? Justify your answer using the graph.
5) From the **Throughput** graph, what was the peak throughput (in Mbps) achieved by the TCP connection, and how does it compare to your nominal ISP bandwidth?